



P Y T H O N

# Lesson 4

*Functions I — Using Built-ins & Modules*

---

Theory · 30 minutes

Hands-on lab · 90 minutes

*Building on Lesson 3: strings, input, and reading errors.*

# What we'll cover in 30 minutes



## Built-in functions

Ready-to-use tools Python gives you for free



## Importing modules

Unlocking extra functionality with import



## The math module

Square roots, rounding, constants, and more



## The random module

Randomness: dice, cards, shuffling



## Function composition

Feeding one function's output into another

## Last time, in Lesson 3...



### Strings

Indexing, slicing, methods



### Input & f-strings

input() + formatting output



### Debugging

Tracebacks & print-debugging

*You've already been calling functions all along — `print()`, `len()`, `input()`, `type()`. Today we go wider.*

## FUNCTIONS

# A function is a mini-machine

It takes in some inputs (arguments), does something with them, and hands back an output (the return value).



*You've used this pattern already: `print()`, `len()`, `input()`, `type()`, `round()`, `int()`, `str()` are all built-in functions.*

# A few more built-ins worth knowing

**max(a, b, ...)**

`max(3, 7, 2) → 7`

**min(a, b, ...)**

`min(3, 7, 2) → 2`

**sum(list)**

`sum([1, 2, 3]) → 6`

**abs(x)**

`abs(-5) → 5`

**sorted(list)**

`sorted([3,1,2]) → [1,2,3]`

**pow(x, y)**

`pow(2, 5) → 32`

*Full list: [docs.python.org](https://docs.python.org) → “Built-in Functions” — bookmark it, you'll return to it often.*

## MODULES

# A module is a file full of ready-made functions

Python's standard library ships with dozens of modules — no installation needed. You just have to import them.



**math**

Square roots, trig, constants



**random**

Randomness: dice, cards, choices



**datetime**

Dates and times



**os**

Files and the operating system

*We'll focus on math and random today — the other two are here to show you the pattern continues.*

# Three ways to import

```
import math
```

*Access everything with a prefix: `math.sqrt(9)`*

```
from math import sqrt
```

*Import just what you need: `sqrt(9)` — no prefix*

```
import random as r
```

*Give it a short nickname: `r.randint(1, 6)`*

# A few math functions you'll use often

**`math.sqrt(x)`**

`math.sqrt(16) → 4.0`

**`math.floor(x)`**

`math.floor(4.9) → 4`

**`math.ceil(x)`**

`math.ceil(4.1) → 5`

**`math.pow(x, y)`**

`math.pow(2, 3) → 8.0`

**`math.pi`**

`math.pi → 3.14159...`

**`math.gcd(a, b)`**

`math.gcd(12, 18) → 6`

# Randomness, on demand

`random.random()`

*→ a float between 0.0 and 1.0*

`random.randint(a, b)`

*random.randint(1, 6) → a die roll*

`random.choice(list)`

*random.choice(["H", "T"]) → a coin flip*

`random.shuffle(list)`

*shuffles a list in place*

*Every run gives a different result — that's the point! You'll simulate dice and cards with these in the lab.*

# Feed one function's output into another

Python evaluates from the inside out — just like nested parentheses in math.

```
round(math.sqrt(50), 2)
```

```
1  math.sqrt(50) runs first → 7.0710678...
```

```
2  round(7.0710678..., 2) runs next → 7.07
```

```
Final result: 7.07
```

# Why use a module instead of writing it yourself?



## Tested & reliable

Used by millions of programs — edge cases already handled



## Fast

Often implemented in optimized C under the hood



## Reusable

Write your logic once, import it anywhere



## A huge head start

Why reinvent square roots when `math.sqrt()` exists?

NEXT: HANDS-ON LAB • 90 MIN

# Now it's your turn

- 1 Build a scientific calculator with math
- 2 Simulate dice rolls & card draws with random
- 3 Compose functions to solve a multi-step problem

*Open the Lesson 4 Hands-On Lab worksheet to get started.*